Lab Cleanup Robot using Mirte Master Platform

AUTHOR:

21st Apr, 2026

Keywords Navigation2, ROS2, SLAM, Mobile$_{M}anipulation$

Contents

In laboratories, items of many kinds may fall onto the floor. While implementations to clean such floors - like robotic vacuum cleaners - already exist, they don't work with trash-separation mechanics and end up disposing of all waste into the same bin. Yet, in laboratories, one may one specific dropped items to be returned and reused, such as electronic components. This project aims to use the open-source MIRTE Master robotic platform to combine both cleaning and reusing, by implementing a sorting system. This project presents an automated trash-detecting and sorting robot, using a 4-DOF robotic arm with gripping end-effector for grabbing, a 2-sided bin for sorting, an RGB-D camera for object-detection, MoveIt! 2 for manipulation and Nav2 combined with SLAM for navigation, built on top of the MIRTE Master platform. During this research, methods for path planning and object detection have been tested, compared and implemented with the goal of making the robot as accurate as possible while maintaining operational velocity.

1 Introduction

In laboratory environments, maintaining a good level of cleanliness is often a challenge. While in modern domestic situations, an autonomous floor-cleaning robot is regularly used nowadays, in laboratory environments this is often not yet the case. In a laboratory, what actually falls onto the floor usually consists of graspable objects, while in domestic spaces these types of robots are generally made to remove dust and hair instead. Hence, a robot for laboratory-cleaning purposes should be able to grab objects instead of dust or hair. But as one may not always want to throw away all objects found in a laboratory setting, one would also want to separate trash from other fallen objects, such as electronics. Hence, the goal for this project was to develop an autonomous laboratory-cleaning robot that can detect, separate and dispose of two different types of objects, all based on the MIRTE Master robotic platform.

This report outlines the roadmap and methodology required to make such a robot. \ Section 1 (Materials) includes information about the hardware and software used to realize the robot. \ Section 2 (Theory) explains in detail how the theory behind the implemented ideas works. \ Section 3 (System Overview) breaks down the implementation of all major aspects of the robot into the robot. Section 4 (Experimental Methods) shows how the implementations are tested. \ Section 5 (Results) covers the results of the tests that have been performed. \ Section 6 (Conclusion), at the end, revisits the purpose of the project and reflects on that using the results that have been obtained and explained in Section 5 (Results).

1.1 Related Works

1.2 Contribution

2 Materials

Within this section, all hardware employed within the MIRTE Master will be displayed and discussed.

2.1 Hardware Specifications

The majority of the hardware used for this implementation of the MIRTE Master stems from the standard components implemented by the MIRTE team. Within this Materials section, below, the list of all hardware parts can be seen, excluding only trivial components such as nuts and bolts.

Category	Components
Chassis	*Panels:* - Left (with OLED-display) and right chassis side panels (3D-printed) - Rear battery bracket panel (3D-printed) - 2x rear Sonar-module panel (3D-printed) - Front RGB-D camera module (3D-printed) - 1.5 mm aluminium: top plate, bottom plate and manipulator-mounting plate (laser-cut) *Electronics:* - Main computer: Orange Pi 3B V1.1.1 - Microcontroller: Raspberry Pi Pico H - MIRTE custom PCB - Micro-SD-card - 12V RGB LED-strip
Manipulation	- Upper arm limb, lower arm limb (3D-printed) - Schoulder joint bracket, wrist joint bracket (3D-printed) - Double gears, bars for 4-bar linkage of gripper, triangle tips for gripper, TPE gripper ends (3D-printed) - Mounting bracket for RGB-camera module (3D-printed) - 5x Hiwonder bus servo-motors - Ball bearing for shoulder rotation joint
Powertrain	- Parkside 12V 5Ah Li-ion battery - Battery-to-circuit connector (3D-printed) - 12V to 5V step-down converter - Wiring, connectors, button and fuse
Locomotion	- 4x geared DC-motors - 4x mecanum wheels - 4x DC-motor brackets (3D-printed)
Distance sensors	- 2D-LiDAR: RPLiDAR C1 - 2x Ultrasonic sensor: HC-SR04
Cameras	- RGBD (depth) camera: Orbbec3D Astra Mini S - RGB-camera: 720p USB camera module

Given that MIRTE doesn't work with large quantities of robots, meaning no standardized parts are available, 3D-printing was a viable choice to establish the necessary parts of the frame using (for this specific project slightly adjusted versions of) the CAD-models provided. These parts can be divided into two groups: chassis and manipulator.

2.2 Chassis

The chassis consists of a top, bottom and manipulator-mounting plate, for which aluminium plates with a thickness of 1.5 mm were used, as well as several side panels. These side panels don't only act as chassis support elements but also as mounting components for several electronics parts. For both of these purposes, PETG was chosen as a good filament to use for 3D-printing the panels. This filament been used for this application due to its relatively high strength, in addition to having enough ductility, it is easy to print and is relatively cheap. The ductility not only makes the robot more resistant to impact due to it being less brittle, it also allows for some slight bending of the chassis which has the added benefit that the wheels are more likely to keep traction on the floor.
The variant of PETG used for this project was translucent which, aside from being aesthetically pleasing, also enables the user to look at the status lights on the inside of the chassis. This would otherwise not have been possible due to the aluminium top and bottom plates. During printing, orange PETG accent lines have also been added to the robot for both aesthetic purposes and it standing out more to people walking by.

2.3 Manipulator

The manipulator consists of several components that can be found within the list at the top of the Materials webpage. This system can be divided into four groups: brackets, limbs, servo-motors and the gripping mechanism. Given that there are four servo-motors, the arm is defined to have four degrees of freedom to be able to reach all places around the robot. There is a fifth servo-motor mounted, but that only actuates the gripping mechanism and therefore doesn't add any degree of freedom to the system. All of this, including the chassis, can be seen in the interactive display near the bottom of the Mechanical overview page.

2.4 Software Specifications

The MIRTE Master robotic platform has free open-source software available for any user to install.
The latest stable release is used in this robotic system. This software comes packaged inside a ROS application Macenski et al. (2022), a standardized framework for developing distributed robotic systems. This allows for the use of a wide variety of cross-compatible plugins and additional software packages, making rapid prototyping and development more efficient.

The particular ROS distribution the MIRTE Master platform implements is ROS2 Humble Hawksbill on Ubuntu 22.04.

2.4.1 SLAM

Before the robot is able to perform any complex task in an environment, the environment must first be mapped. For this, SLAM (Simultaneous Localization and Mapping) is used. This way the robot can dynamically update its environment based on measurements from its LiDAR scanner. The robot created an occupancy grid map as an image while estimating the robot's pose.

To implement SLAM into the MIRTE Master robot, SLAM Toolbox is used (Macenski and Jambrecic (2021)). This software package was chosen due to its native ROS2 support and good integration with other ROS2 packages. Slam Toolbox allows for straight forward modification of mapping behavior using the set of parameters it provides. In the case of the MIRTE Master, there are several projects that have implemented SLAM Toolbox, so finding a ready-to-use set of SLAM parameters for this robot is relatively simple. The Mirte Navigation ROS package is used for configuration files and SLAM parameter settings. It was assumed the robot would only navigate in unknown environments without predefined or reoccurring maps. Therefore localization (using AMCL) was disabled in the application.

2.4.2 Manipulation

While the arm on the MIRTE Master can be controlled in joint-space, the implementation relies on task-space (cartesian-space) control over the end effector position.
The manner which in motion planners are typically implemented requires the integration of several complex subsystems like inverse and forward kinematic solvers, trajectory planners and path planners. To accomplish the goal of task-space control over the arm, MoveIt 2 (Coleman et al. (2014)) was used.

2.4.3 Navigation

Robot navigation also has a challenge analogous to that of robot manipulation, namely that of workspace control. The robot must be able to navigate to or through a set of waypoints given in the coordinate system of the map, while also implementing a real-time controller for obstacle avoidance. Similar to MoveIt 2, Nav2 is used as a solution to this problem (Macenski et al. (2020)). Aside from path planning and real-time control, Nav2 also provides a Costmap.

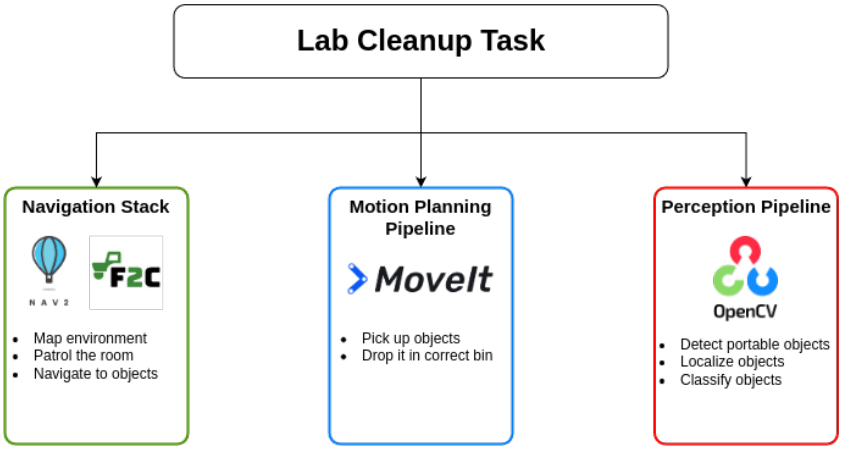
2.4.4 Vision

3 System Overview

The setup consists of a mobile manipulator (MIRTE Master) tasked with autonomously exploring an indoor laboratory environment, identifying and localizing objects, distinguishing between electronics and other objects and sorting these objects accordingly.

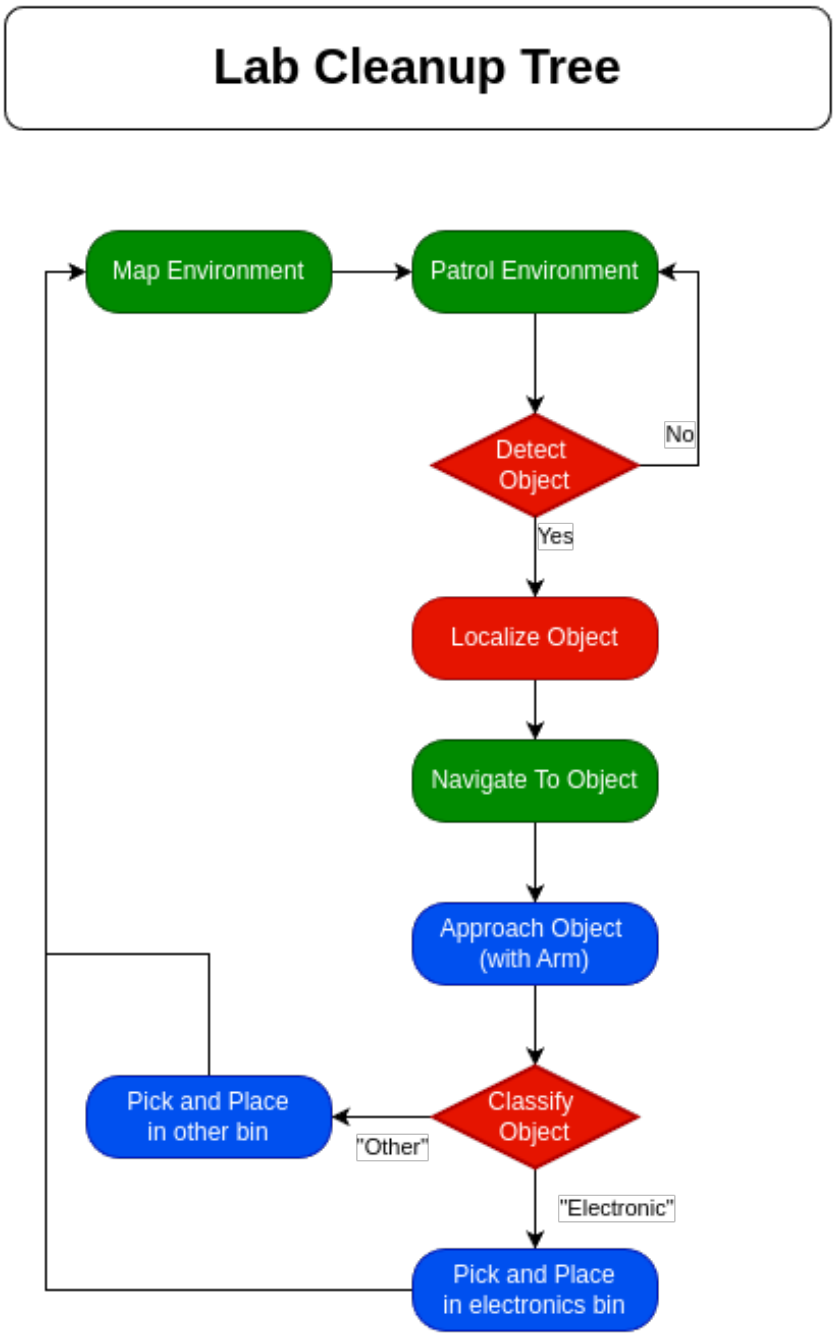
The main task of the robot can be broken down into sub-tasks, these then fit into specific niches of the entire system architecture:

1. **Motion planning**
2. **Perception**
3. **Navigation**
   Figure **??** below showcases this idea.



The scope of this project includes how different mapping, navigation and perception approaches influence the performance of the whole system. The highest performing combination of approaches is then chosen for use in the respective subsystems.

The system level strategy dictates how the robot behaves in a given situation. The tasks described above are used as a guideline to construct the full system level strategy. The standard in robotics for such navigation and perception heavy tasks is to use a global behavior tree that describes how the robot ought to behave. This system level architecture, as shown in Figure **??**, is also used here to describe and execute the actual cleaning strategy.


Lab Cleanup Tree
Map Environment
Patrol Environment
Detect Object
No
Yes
Localize Object
Navigate To Object
Approach Object (with Arm)
Classify Object
"Other"
Pick and Place in other bin
"Electronic"
Pick and Place in electronics bin

3.1 Navigation

The navigation stack of this robot is responsible for two main tasks:

- Mapping
- Coverage

3.1.1 Mapping

Before the robot can navigate properly through the environment and ensure proper coverage, this environment must first be known. That is where mapping approaches can be helpful. Several advanced and specialised mapping approaches already exist, but in this paper only one was considered: 'frontier based mapping'. *Once the environment is mapped sufficiently, the robot is allowed to navigate the space and propagate its behavior further down the tree.*

Unlike less advanced alternatives, frontier based mapping allows the robot to start navigating the space and propagate its behavior further down the tree as soon as the environment is mapped sufficiently.

3.1.2 Coverage Planners

This section presents the algorithms employed for the discovery of portable objects. While a comprehensive technical analysis of these algorithms is beyond the scope of this work, they are described with sufficient detail to enable understanding of their application within our framework. For in-depth technical details and formal derivations, the reader is referred to the original publications cited herein. To demonstrate the different navigators, a costmap of the Pulse Breakout room is used. This map was obtained during testing of the charting methods.

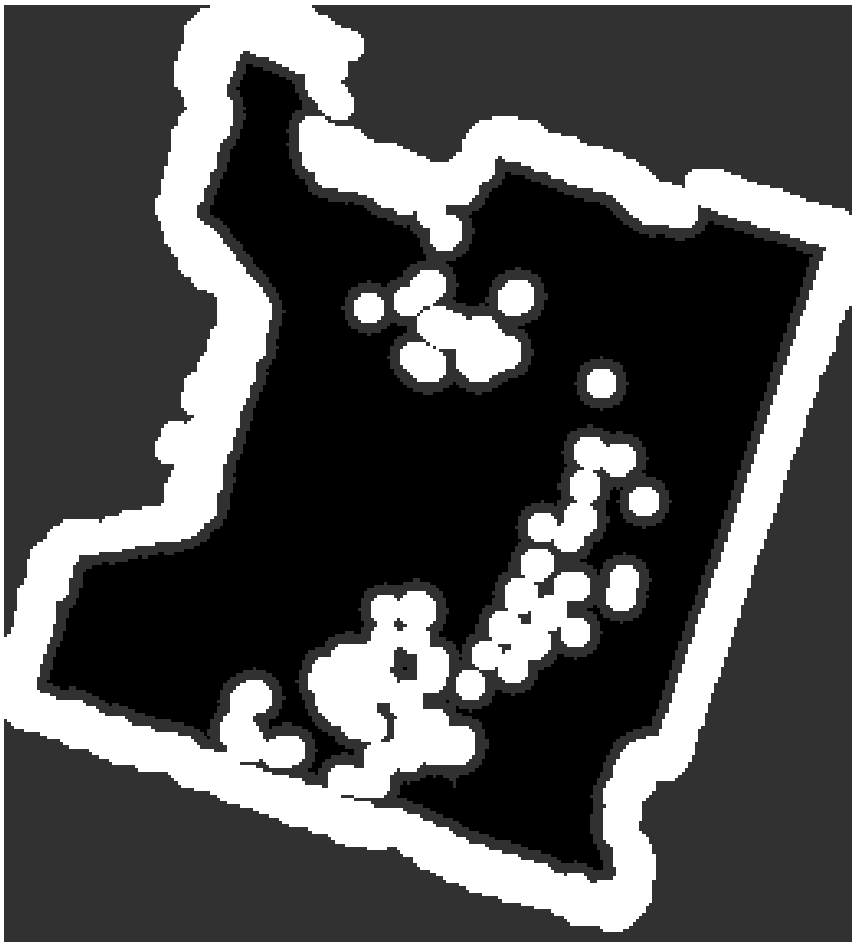


Figure 1: Initial costmap of the Pulse Breakout room used to evaluate the coverage planners.

Nav2 is used for low level motion planning and execution aswell as realtime control and costmap generation.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display

def interactive_path_slider(
    image_path,
    csv_path,
    title="Path Viewer",
    color=(0, 255, 255),
    thickness=1,
):
    """
    Visualize a path stored in CSV over an image.
```

```
Parameters
- - - - - - - - - -
image_path : str
    Occupancy map image.

csv_path : str
    CSV with columns:
        x,y
        74,19
        75,20
        ...
"""

plt.ion()

# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Load image
# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

map_img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if map_img is None:
    raise ValueError(f"Could not load image: {image_path}")

# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Load path
# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

path_pixels = np.loadtxt(
    csv_path,
    delimiter=",",
    skiprows=1,
    dtype=np.int32,
)

if path_pixels.ndim == 1:
    path_pixels = path_pixels.reshape(1, 2)

# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Draw path
# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

vis = cv2.cvtColor(map_img, cv2.COLOR_GRAY2RGB)

cv2.polylines(
    vis,
    [path_pixels],
    isClosed=False,
    color=color,
    thickness=thickness,
)

# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Create figure
# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

fig, ax = plt.subplots(figsize=(8, 8))
```

```
ax.imshow(vis)
ax.set_title(title)
ax.axis("off")

robot_dot, = ax.plot(
    path_pixels[0, 0],
    path_pixels[0, 1],
    "o",
    color="yellow",
    markersize=10,
)

# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
# Slider
# - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

slider = widgets.IntSlider(
    value=0,
    min=0,
    max=len(path_pixels) - 1,
    step=1,
    description="Waypoint",
    continuous_update=True,
    layout=widgets.Layout(width="800px"),
)

def update(change):

    idx = change["new"]

    x = path_pixels[idx, 0]
    y = path_pixels[idx, 1]

    robot_dot.set_data([x], [y])

    fig.canvas.draw_idle()

slider.observe(update, names="value")

display(slider)
plt.show()
```

Preprocessing Before the map can be used to plan a path, a common preprocessing algorithm can be abstracted. Coverage path planners generally use either a binary map of the free space, (free=1, occupied=0) or the polygons constructed from the boundaries of the free space.

The input to the algorithm can be any 2D image displaying the free and occupied pixels. In this case the input will be the costmap directly. First the costmap is thresholded to create a Figure 1, which is the first output.

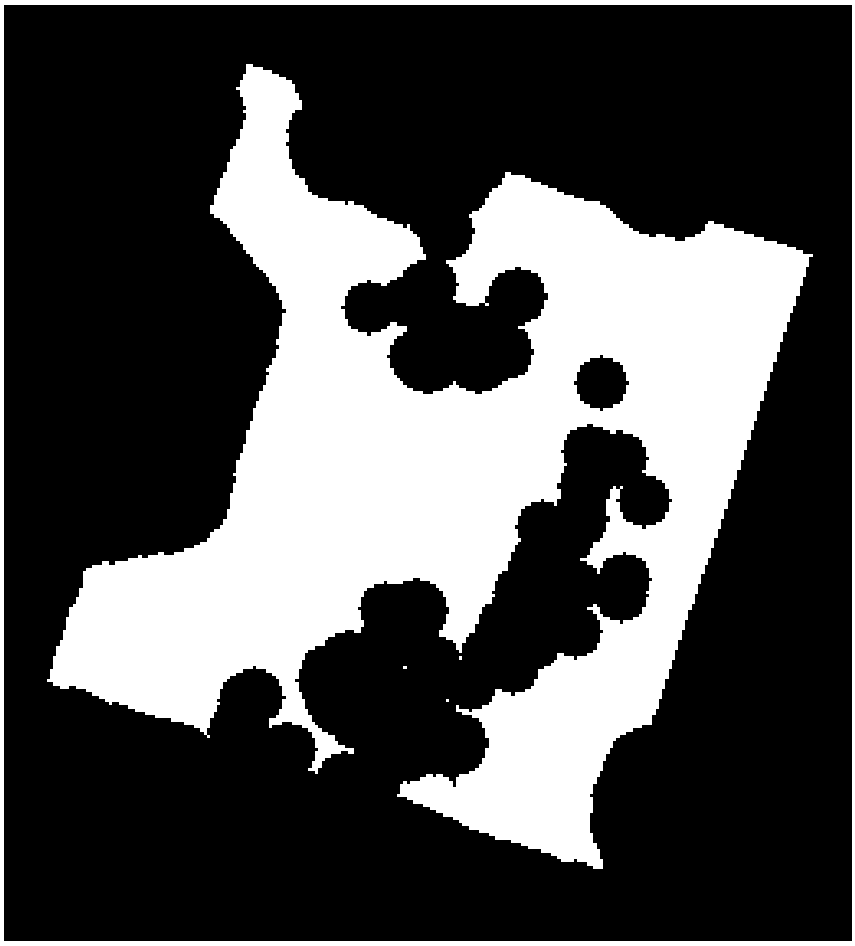


Figure 2: Binary costmap generated by thresholding the Nav2 costmap into free and occupied space.

Then using this binary costmap, the contours Figure 3 are extracted.

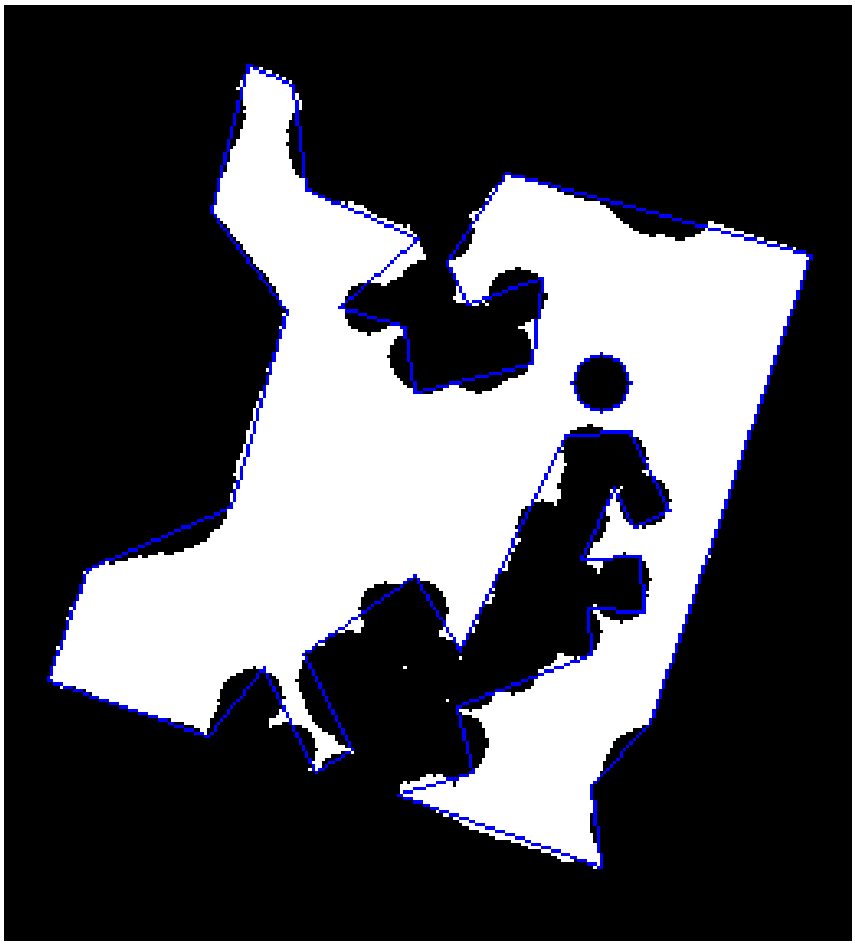


Figure 3: Contours extracted from the binary costmap representing the boundaries of free space.

Morphology-Based Skeleton Path Planner The first implemented method, based on the work of (Becoy et al., 2025), (Niu et al., 2025), derives a coverage path from the morphological skeleton of the free space. See the algorithm below.

Input: Occupancy grid $\mathcal{M}$, robot start position $\mathbf{p}_0 \in {}^2$
Output: Ordered waypoint segments $\mathcal{P} = [P_1, P_2, \ldots, P_k]$

---

1. $B \leftarrow$ threshold($\mathcal{M}$, lethal_threshold)
2. Extract contours from B, group by containment into polygon groups G_{poly}
3. **For each** polygon group $g \in G_{poly}$:
   (a) $S \leftarrow$ medialAxis(B) *(skimage skeletonize)*
   (b) $W \leftarrow$ skeletonToWorldCoords(S)
   (c) $G_{graph} \leftarrow$ buildKDTreeGraph(W, radius $= r_{path}$)
   (d) bridgeDisconnectedComponents(G_{graph})
   (e) $L \leftarrow$ findLeafNodes(G_{graph}) *(nodes with degree 1)*
   (f) $v \leftarrow$ nearestLeaf($\mathbf{p}_0$, L)
   (g) $\mathcal{Q} \leftarrow [v]$, $V_{visited} \leftarrow$
   (h) **While** $V_{visited} \neq L$:
      i. $V_{visited} \leftarrow V_{visited}\{v\}$
      ii. $r \leftarrow$ nearestLeafByGraphDistance(v, $LV_{visited}$, G_{graph})

iii. $\pi \leftarrow$ shortestPath(v, r, G_{graph})
iv. append π to $\mathcal{Q}$
v. $v \leftarrow r$

(i) $P_i \leftarrow [W[u] \,\forall\, u \in \mathcal{Q}]$
(j) append P_i to $\mathcal{P}$, set $\mathbf{p}_0 \leftarrow P_i[last]$

4. sanitiseWaypoints($\mathcal{P}$, $\mathcal{M}$)
5. **Return** $\mathcal{P}$

The free space is extracted using the methods discussed in the previous section. The polygons are used and converted to a binary map as to not produce too many branches for the robot to navigate through. This map is then skeletonized to extract waypoints, see `Figure {number} <fig-skeleton_free>`.

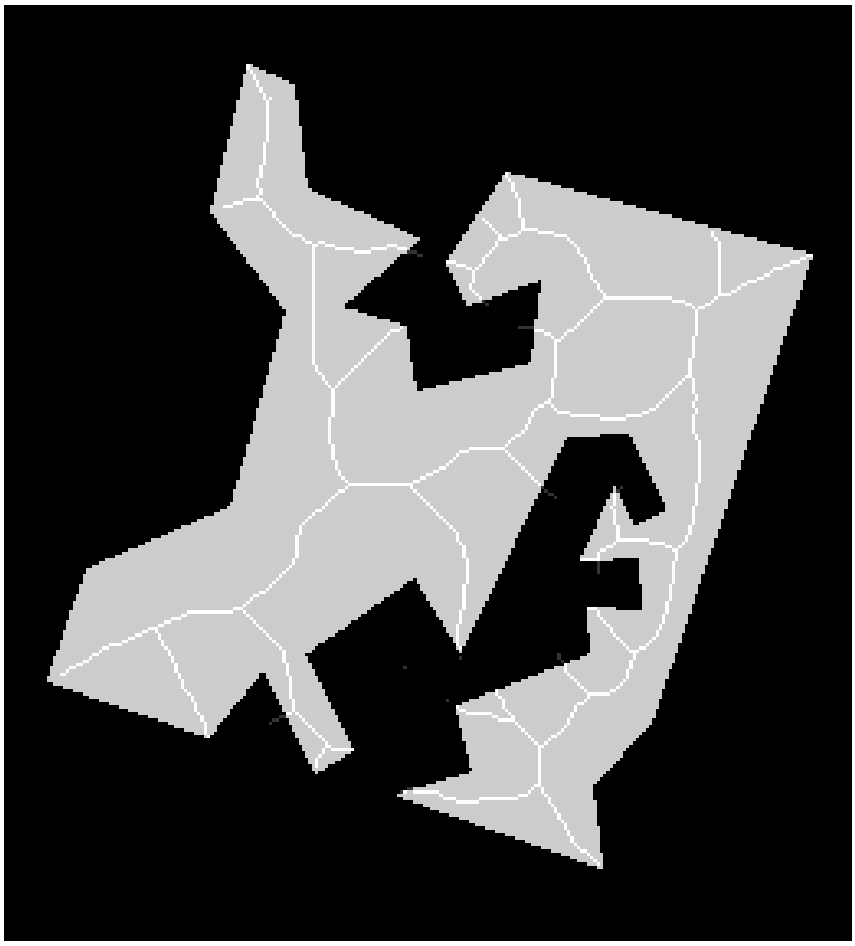


Figure 4: Morphological skeleton extracted from the free-space representation of the environment.

Once the waypoints are obtained, a graph is constructed and leaf nodes are extracted. In the graph leaf nodes are defined by there single connection to the rest of the skeleton, these are seen as endpoints of the path.

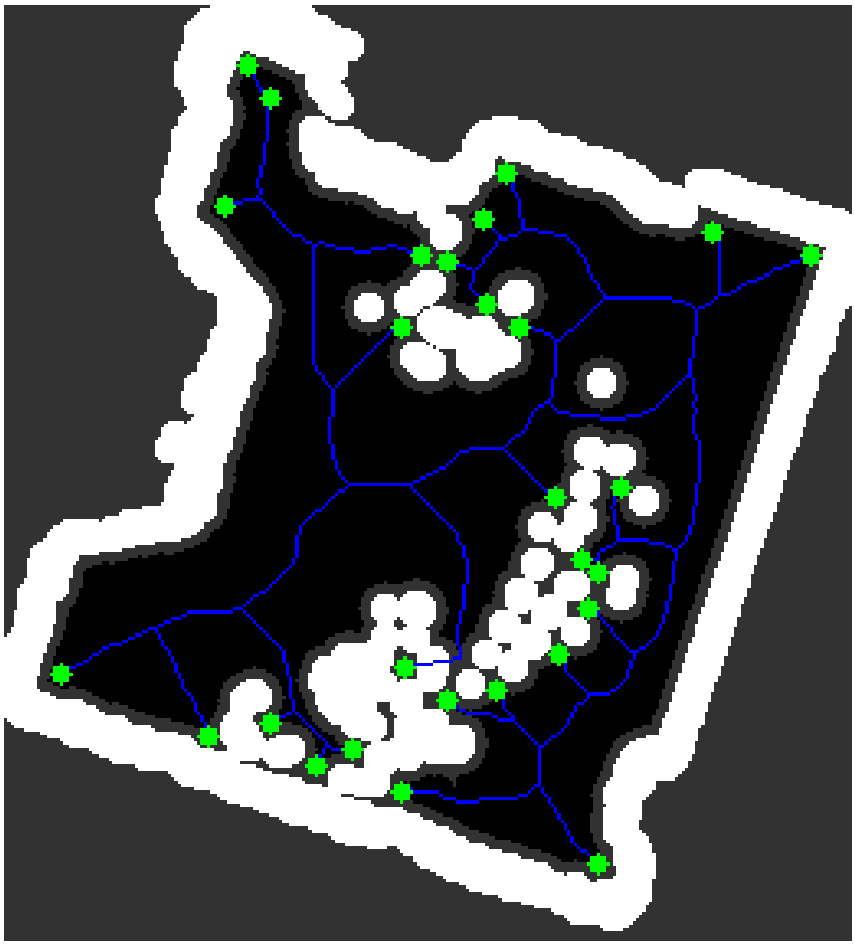


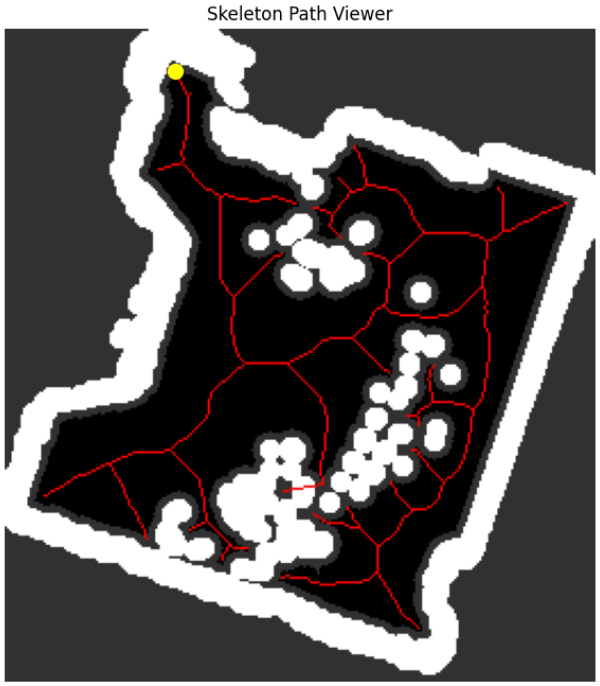
Figure 5: Leaf nodes identified on the skeleton graph. These nodes represent endpoints used during path generation.

Finally, starting from the closest leaf node to the robot, the full path is planned by recursively tracing the graph from the current leaf node to the nearest unvisited leaf node. Use the slider below to visualize the full path

```
%matplotlib widget

interactive_path_slider('figures/navigator_results/Initial_Costmap.png',
                        'figures/navigator_results/skeleton_path.csv',
                        "Skeleton Path Viewer",
                        (255, 0, 0), 1)

IntSlider(value=0, description='Waypoint', layout=Layout(width='800px'), max=1862)
```



Grid Based Spanning Tree Planner Secondly a grid based spanning tree algorithm is used to plan a full coverage path over the map. This planner is based on the work of (Gabriely and Rimon, 2001)

Input: Occupancy grid $\mathcal{M}$, robot start position $\mathbf{p}_0 \in {}^2$, scale $s = 0.06$
Output: Ordered waypoint segments $\mathcal{P} = [P_1, P_2, \ldots, P_k]$

---

1. $B \leftarrow$ threshold($\mathcal{M}$, lethal_threshold)
2. $B_{down} \leftarrow$ resize(B, scale $= s$, interpolation $=$ NEAREST)
3. $G \leftarrow$ gridGraph(B_{down}) *(4-connected)*
4. Remove all edges (u, v) where $B_{down}[u] = 0$ or $B_{down}[v] = 0$
5. $T \leftarrow$
6. **For each** connected component $\mathcal{C} \in G$:
   (a) $T \leftarrow T$ DFS_tree($\mathcal{C}$)
7. **For each** edge $(u, v) \in T$: *(edge subdivision)*
   (a) $m \leftarrow \frac{u+v}{2}$
   (b) replace (u, v) with edges (u, m) and (m, v)
8. $P_{perim} \leftarrow \mathbf{0}^{H \times W}$, $r \leftarrow \lfloor \frac{1}{4s} \rfloor$
9. **For each** node $p \in T$:
   (a) $p_{img} \leftarrow \frac{p+0.5}{s}$
   (b) drawFilledRect(P_{perim}, centre $= p_{img}$, halfsize $= r$)
10. $\mathcal{C} \leftarrow$ findContours(P_{perim}, RETR_EXTERNAL)

11. **For each** contour $c \in \mathcal{C}$:
    (a) $W \leftarrow$ pixelToWorld(c)
    (b) **If** $|W| < 2$: skip
    (c) $idx \leftarrow \arg\min_i \|W[i] - \mathbf{p}_0\|_2$
    (d) $P_i \leftarrow [W[idx], \ldots, W[-1], W[0], \ldots, W[idx-1]]$
    (e) append P_i to $\mathcal{P}$, set $\mathbf{p}_0 \leftarrow P_i[last]$
12. sanitiseWaypoints($\mathcal{P}$, $\mathcal{M}$)
13. **Return** $\mathcal{P}$

As opposed to the Skeleton planner, this planner makes use of the binary costmap directly. Using this binary costmap, it is further discretized to a grid which is the fraction of the resolution of the original costmap. `Figure {number} <fig-spanning_grid>`.

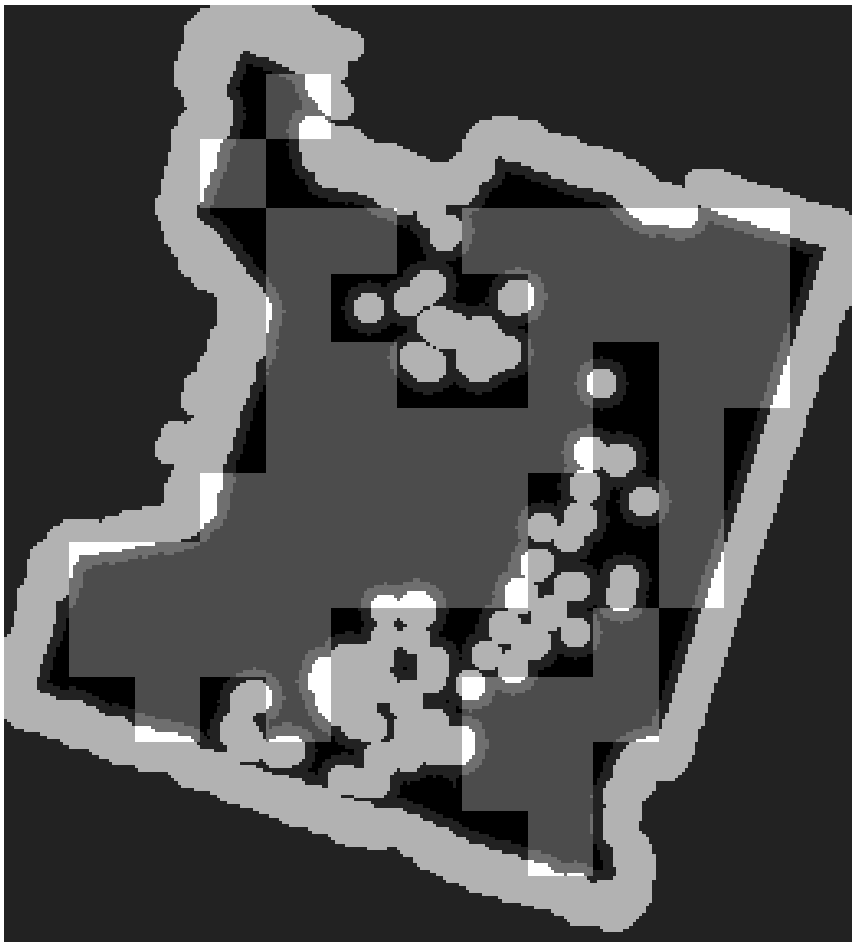


Figure 6: Downsampled free-space grid used to construct the spanning tree graph.

A degree first search (DFS) is conducted over the free cells, from which sparse waypoints are created.

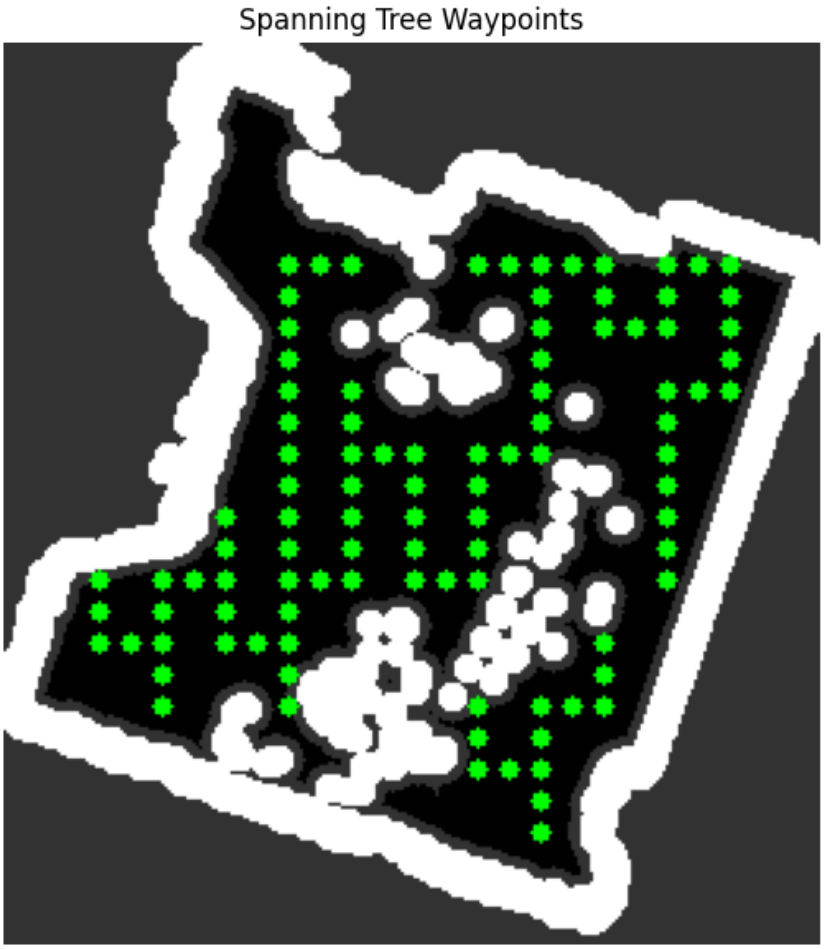


Figure 7: Depth-first search spanning tree generated over the free-space grid.

A black image, of the same resolution as the original costmap is then created and white squares, with half the cell width, are placed in the center of each cell and in between each cell. Waypoints are created from the contours of this tree. These waypoints represent a circumnavigation around the DFS tree.

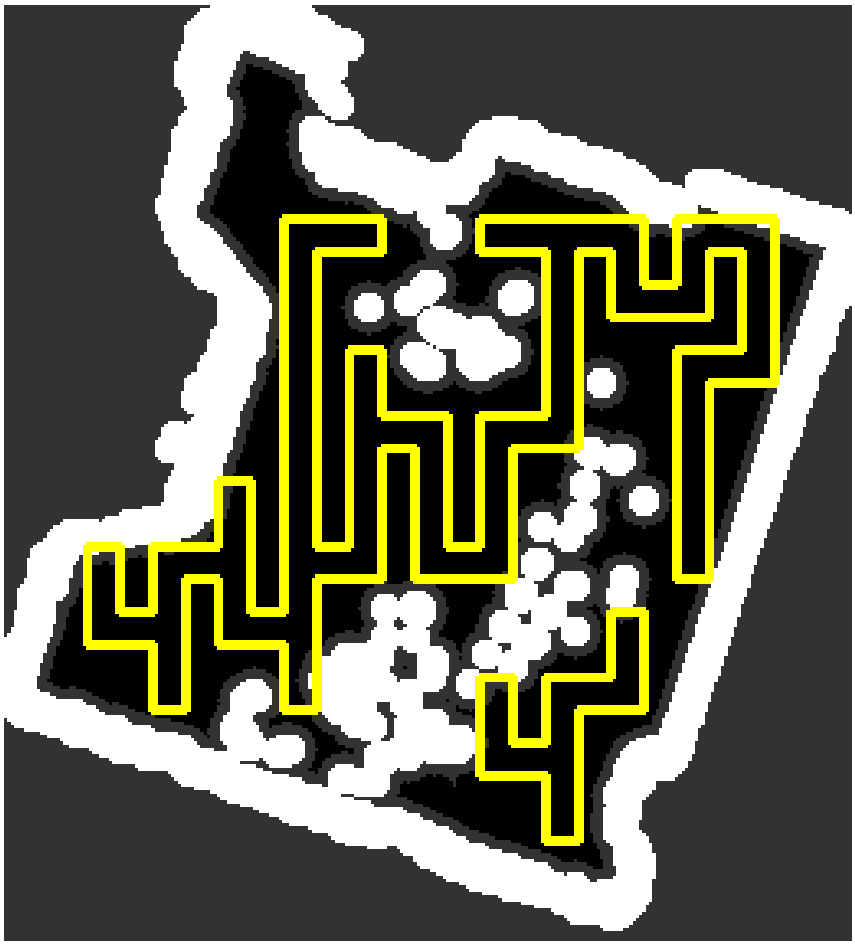


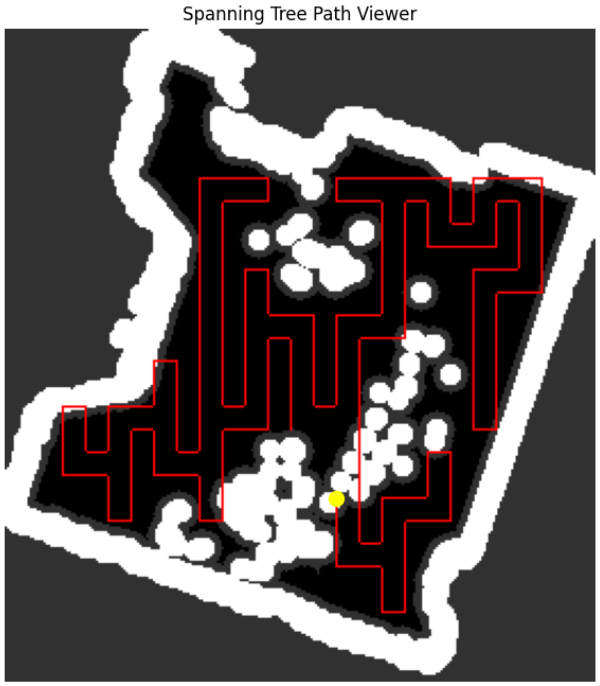
Figure 8: Coverage waypoints generated by tracing the contour around the spanning tree structure.

A segment from this circumnavigation is then removed from the path

```
%matplotlib widget

interactive_path_slider('figures/navigator_results/Initial_Costmap.png',
                        'figures/navigator_results/spanning_tree_path.csv',
                        "Spanning Tree Path Viewer",
                        (255, 0, 0), 1)

IntSlider(value=0, description='Waypoint', layout=Layout(width='800px'), max=109)
```


Spanning Tree Path Viewer

3.2 Mechanical

3.2.1 Original MIRTE Master Platform

The project is based on the MIRTE Master V2. The platform consists of a mobile base propelled by four individually driven mecanum wheels and a robotic arm, allowing the robot to navigate through an environment and interact with objects.

The original MIRTE Master hardware consists of three main parts:

1. **Mobile base**
2. **Electronics**
3. **Robotic arm**

The mobile base utilizes four geared DC motors with Hall encoders, connected to 97 mm mecanum wheels. This allows the robot to move in multiple directions without having to turn first, which is useful for navigating in small indoor spaces. The base also contains various sensors, such as rearward facing ultrasonic sensors, an IMU, a 2D LiDAR and an RGBD (depth) camera mounted on the robotic arm.

The electronics of the original platform are based on an Orange Pi 3B and a Raspberry Pi Pico H. The Orange Pi functions as the main computer, while the Pico is used for lower-level control tasks such as controlling the motors and general interfacing. The MIRTE Master also includes a custom-made PCB to connect the motors, sensors, and power system in a structured manner. The main power source for the MIRTE Master is a 5Ah 12V Lion Parkside drill battery. No significant changes were made to the electronics used for this project.

The robotic arm is mounted on top of the mobile base and has four degrees of freedom, which are driven by four serial bus servo motors of varying strengths. A gripper and RGBD camera are mounted at the end of of the arm for object scanning and manipulation.

3.2.2 Hardware Components

The most important original hardware components can be seen in Materials.

These components provide a useful base platform for a mobile manipulation task. However, for this project, the original configuration needed changes. The robot had to be adapted for a laboratory-cleanup task, where it should be able to move around in a laboratory environment, detect waste on the floor, and eventually collect and move objects into bins.

3.2.3 Design Goals for the Modified Robot

The goal of the hardware modifications was to make the MIRTE Master more suitable for collecting and transporting small waste objects. This required changes to the chassis and and gripper design.

The main design goals were:

- Add space for collecting and separating waste.
- Mount the camera in a useful position for object recognition.
- Improve the battery mounting system.
- Improve cable management.
- Modify the gripper so it can pick up waste objects more reliably.

3.2.4 Chassis Modifications

The original MIRTE Master chassis was designed as a general-purpose mobile manipulator base. For this use case, the robot also needed to carry waste bins and support the manipulator during picking tasks. Therefore, the chassis was modified to improve strength and usability.

The chassis consists of six side panels and a top and a bottom plate. In the original model, the top and bottom plates were made out of clear PMMA. While PMMA allows the user to see the status of the electronics clearly

by looking at the LEDs on the inside, it also has a tendency to shatter easily if bent too much. During early testing of the MIRTE Master it has already shown that the PMMA was not a good choice for any type of loading resulting in shattered PMMA. Therefore a change to the material of all plates from PMMA to aluminium with a similar stiffness was essential, the difference of this being that aluminium can bend more without shattering. To still be able to see the status of the electronics by the LEDs, the six side panel elements were 3D-printed using a slightly translucent material.

Two separate waste bins were added to the robot. These bins allow the robot to collect, store and separate objects after picking them up.

3.2.5 Gripper Modifications

The gripper is an important part of the laboratory-cleanup robot because it directly interacts with the objects that need to be collected. The original MIRTE Master gripper is useful as a general robotic gripper, but the cleanup task requires it to handle different small objects reliably.

The original gripper geometry is suitable for holding small and large objects, however, the gripping surface is not. Electronic trash tends to have small contact points for the gripper, so good gripping performance is a must-have. For improving the grip on objects, the contact patches are layered with a layer of soft 83A TPE, coated with an additional layer of Bison Rubber anti-slip coating.

During the initial testing phase of the RGBD camera, it was found that the mounting location for it was too low to the ground for it to be able to confidently display any accurate results regarding the distance towards objects. While this was the intended way for the camera to be implemented, for this project it was a better plan to mount this RGBD camera onto the end of the arm, right above the gripper. This gives it elevation which makes it able to look slightly more downward and therefore it could more accurately display distances. This also makes the need for the secondary RGB camera mounted onto the gripper insignificant, as the RGBD camera can also be used for object detection.

3.2.6 Summary of Hardware Changes

Subsystem	Original design	Modified version
Chassis	Standard MIRTE Master frame with PMMA	Strengthened top and bottom plates with aluminium
Storage	No dedicated cleanup storage	Two separate waste bins added
Camera	RGBD camera mounted at the front of the chassis	RGBD camera mounted on the gripper
Wiring	Standard cable routing	Slightly improved cable management
Gripper	Original MIRTE gripper	Improved gripping surface

3.2.7 Current Design

The complete model can be explored by using the interactive 3D interface below (Mechanical).

```
from IPython.display import HTML

HTML("""
<script type="module" src="https://unpkg.com/@google/model -viewer/dist/model -viewer.min.js"></scri

<model -viewer
    src="https://matt -rbt.github.io/Lab -Cleanup -Robot -using -the -Mirte -Master -Platform/models/
    camera -controls
    auto -rotate
    exposure = 0.4
    style="width: 640px; height: 640px; background: #d1d9e6;">
</model -viewer>
""")
```

```
# IMPORTANT:
# The model path is an absolute GitHub Pages URL on purpose.
# A relative path like "./models/floor_with_cubes.glb" works locally in VS Code,
# but fails on the deployed MyST site because the page is served from /coverageplanners/.
# Do not change this to a relative path unless you also test the deployed site.
```

3.3 Manipulation

This section covers the articulation of the robotic arm that sits atop the MIRTE Master. Since there is hardly any competition in the space, the market leader, MoveIt 2, will be used to move the arm. MoveIt 2 provides a well-documented API and directs the vast majority of computations regarding (inverse) kinematics of robotic arms and the trajectory planning thereafter (source).

The package is already installed on the MIRTE Master and has previously been used semi-successfully to move MIRTE's arm (source). However, this implementation resulted in the end effector (the tip of the arm; the wrist) being positioned more than 10 cm off-target. For a machine needing to pick up objects smaller than 6 cm, this is unacceptable. Therefore, an improved version is required. To this end, the designed behaviour contains a multitude of improvements and concessions:

3.3.1 Changes

- The previously missing configuration file, `moveit_cpp.yaml`, which contains additional parameters (specifically for OMPL, the motion planner), was created and added.
- To accommodate the fact that MIRTE's arm can only move with four degrees of freedom (DOF) instead of the six assumed by MoveIt 2, planning is performed strictly for position, ignoring any orientation component of a goal pose entirely. Together with the first point, this made it possible to use more precise methods from the Move Group Interface to plan and execute movements instead of `setApproximateJointValues()`. The resulting positional accuracy is in the order of millimetres, although alignment of the gripper must be configured separately.
- Functionality has been added to ensure that the gripper points the depth camera at a downward angle with respect to the ground to ensure that the depth camera can correctly detect the floor itself and possible objects on the floor. When an object is detected and needs to be classified, the robot points the depth camera downward with respect to the world axes for the object classification. This ensures that any approached object is directly below the camera. As a result, the gripper camera has a constant and uniform background, increasing classification confidence levels.
- An action server has been developed that provides feedback to a potential action client. The server can receive goals in several formats:
  - By specifying the name of a predefined joint state defined in the robot's description files.
  - By supplying a goal pose relative to MIRTE's frame `base_link`.

  The action server moves the arm in a way that satisfies the goal, if possible. The wrist joint can be moved separately to allow fine adjustment of the gripper's orientation. Similarly, the gripper can be controlled either by specifying a named state or by directly assigning a desired clamping angle. The gripper will move accordingly, if possible.

Together, these modifications facilitate the movement of the robotic arm for this specific project and enable the creation of additional custom manipulation modules for future projects.

3.4 Perception

Perception is responsible for object detection, classification and spatial localization. For this study the object detection and localization approaches were limited to two. A **2D detection and depth projection** approach and 3D segmentation with **point cloud clustering**. As for classification methods, this study evaluates a classical cv approach, as well as a classification model based approach.

Originally, objects where categorized into:

1. Graspable vs non-graspable
2. Electronics vs non-electronics

However, due to the limitations of the hardware of the standard MIRTE Master package (relatively low-quality camera and limited computing power) the scope got reduced to:

1. Graspable vs non-graspable
2. Colourful vs greyscale

3.4.1 Perception 2D

For determining the type, and possibly the distance of objects, a YOLO26 ('You Only Look Once-26') object classification model was used. A dataset was trained by recording videos using the USB Camera included in the MIRTE Master hardware package, of which one in every fifteen frames got extracted to use for the training dataset. A figure of four frames in different environments is shown below.

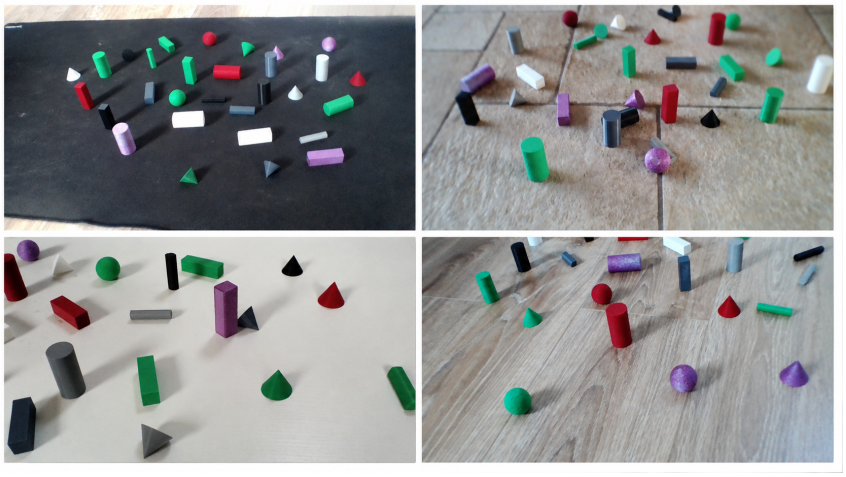


Figure 9: Figure of four individual frames.

Why YOLO26 was used: The YOLO object-detection family is well established, with YOLO26 being the most recent generation. YOLO26 was chosen because the robot has to detect and classify objects from a live camera on the MIRTE Master platform, which has limited processing power as it uses an OrangePi 3B which has limited computational capabilities. Object detection with limited processing power requires a lightweight model with fast inference times while still maintaining accuracy. YOLO26 is suitable for this because it was designed with deployment on low-power devices in mind. It changed the post-processing pipeline to NMS-free inference, simplified bounding-box predictions and supports deployment formats commonly used for low-power devices (Sapkota et al., 2026). In this case, the RK3566 NPU of the OrangePi 3B can be used for improving the performance, reducing the total inference time (Limaran et al., 2026). For this model, the Nano variant of YOLO26 was chosen to minimize inference times. Another benefit of the newer YOLO26 variant over the older variants is that YOLO26 allows for oriented bounding boxes. Unlike standard bounding boxes, oriented bounding boxes also estimate the rotation of recognized objects (Sapkota and Karkee, 2026). This could be useful for complex grasping tasks where orientation of the gripper is crucial. However, this project only uses standard object detection.

Training Training was done manually using Label Studio. Initially, roughly 280 frames were captured in four different environments. Due to a large amount of blurring within the images, about 100 pictures were deleted. Of the remaining 180 pictures, 60 pictures were imported into Label Studio and labelled per colour: green, red, purple, white, light-grey, dark-grey and black. In total, 1206 bounding boxes were drawn. A test of the perception model performed during training is shown in the figure below.

Figure 10: Figure of tests performed during training.

The curves in the figure below show that the model converged during trading. This means that the model actually learns while training instead of making random or unstable decisions.

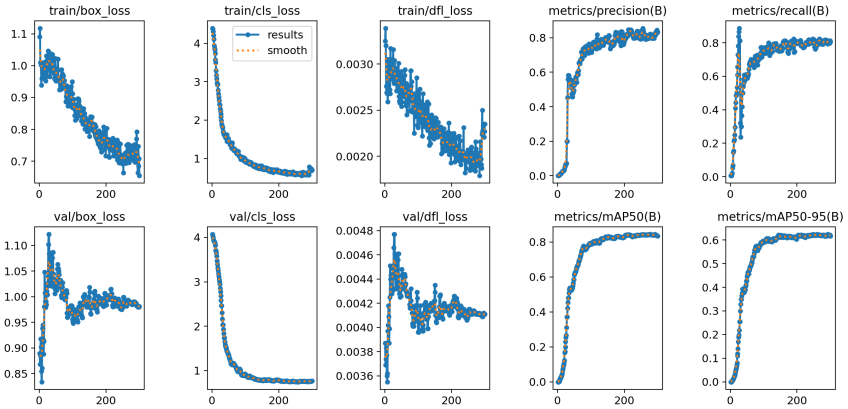


Figure 11: Curves of training results per epoch.

The normalised confusion matrix shows how well classes are recognized by the model. As shown in the figure shown below, most classes are classified correctly, as indicated by the strong diagonal values. However, light-grey objects were among the least correctly identified objects, this is a result of exposure and lighting changing enough to where the model will classify them as background. Some of objects were missed as background, a small amount of objects were misclassified, and some objects like dark grey and white where confused with light-grey.

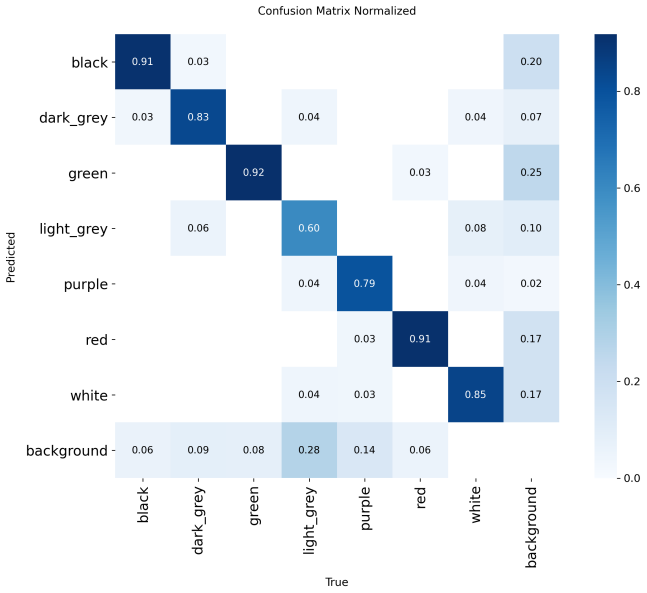


Figure 12: Normalized confusion matrix.

A figure of the F1 confidence curve is shown below. This curve shows how the F1 metric, which combines precision (how many objects where correctly predicted) and recall (how many objects where correctly found), changes with the confidence threshold of the model. The curve shows that a good starting confidence threshold would be 0.3-0.35.

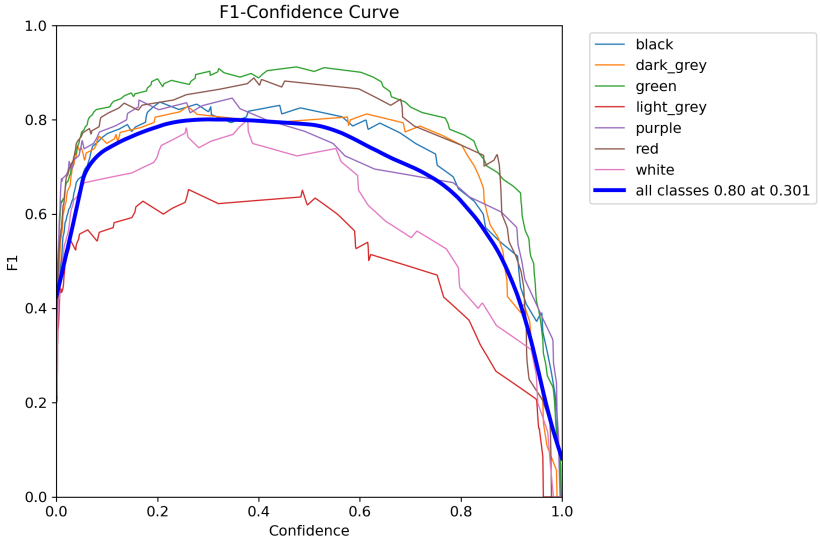


Figure 13: BoxF1 curve

3.4.2 Perception 3D

Point Cloud-based Object localisation: To determine the exact location of graspable objects around the MIRTE Master, the data gathered and transmitted by the camera must be interpreted. One effective method for achieving this is through the use of point clouds. As the name suggests, point clouds are collections of points that represent a three-dimensional space. These points may contain a variety of information, but the most important characteristic of a point during this project is its position, represented by its x, y, and z coordinates.

Detecting planes will be done using the Random Sample Consensus model, or RANSAC for short. This algorithm picks a number of points from the point cloud at random and fits a plane through them. It then evaluates which points in the point cloud are within the given distance from the plane (Fischler and Bolles, 1981). With this method, point clouds can easily and quickly be filtered such that only the important parts, namely the clusters of points representing small objects on the ground, remain.

Next, an algorithm for discovering clusters is used called DBSCAN (Density-Based Spatial Clustering of Applications with Noise). The model evaluates the remaining points in the point cloud and determines the cluster to which they belong (Ester et al., 1996). Afterwards, the identified clusters can be converted into bounding boxes, deciding an object's position and orientation, as well as its dimensions.

Below, figures Figure 14 and Figure 15 show the input to output the constructed point cloud pipeline.

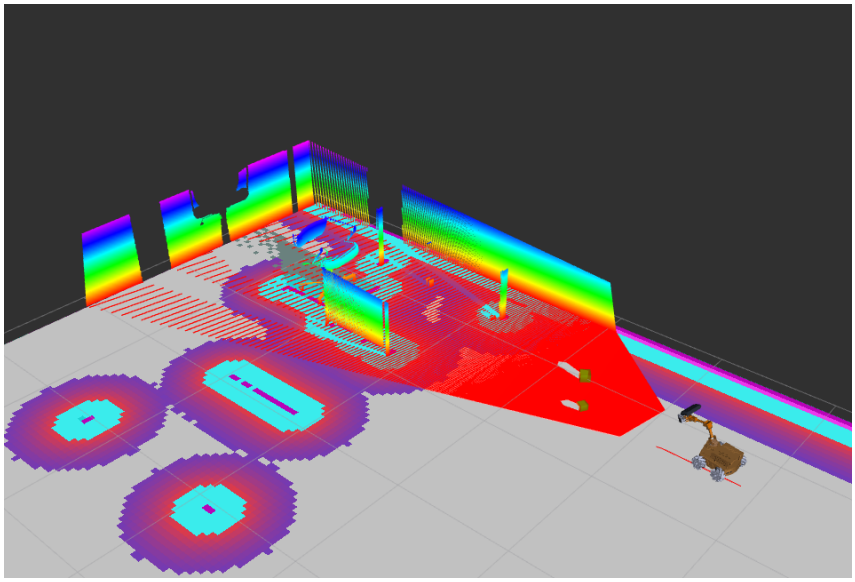


Figure 14: The point cloud as received from the camera topic

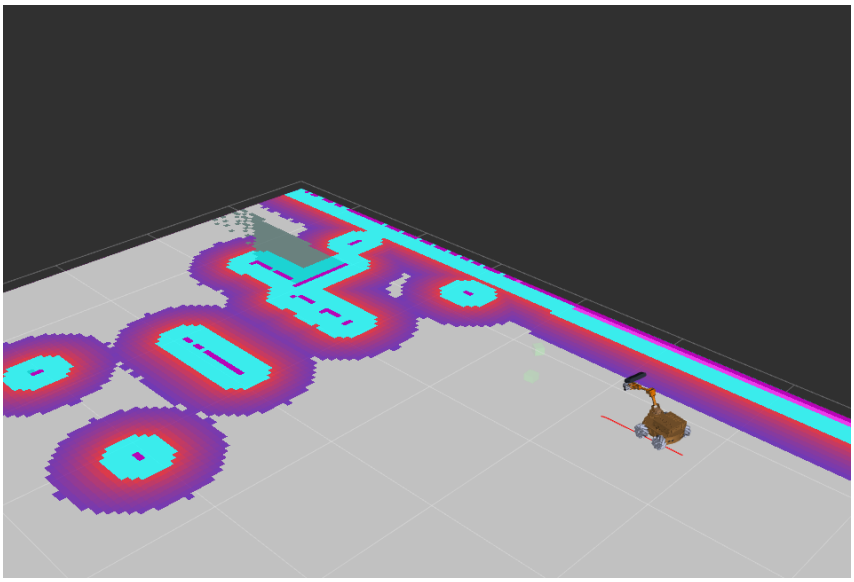


Figure 15: The resulting bounding boxes

Any objects that are in the costmap are excluded, resulting in only these bounding boxes remaining.

4 Methodology

This study follows a factorial experimental design, analyzing the interplay between perception, coverage, and system-level decision-making.

Three independent variables are defined:

- **Detection Method:**
  - Point cloud clustering
  - 2D depth projection
- **Coverage Strategy:**
  - Morphology-based skeleton coverage
  - Spanning tree coverage
- **System-level Decision-making:**
  - Systematic coverage
  - Clean as-you-go

This results in a total of 8 configurations to be tested and compared.

4.1 From Simulation to Real-World Application

For the transition from simulation towards real-world testing, the codebase needed to be changed. The launch files were changed so that Gazebo was no longer used and `use_sim_time` parameters were set to `false`. To reduce the risk of collisions, the inflation radius and the robot radius in the Nav2 parameters were increased. Another change was the migration of processing tasks from the OrangePi 3B to a laptop connected to the MIRTE. Finally, all the sensors were tested and it was concluded that the depth camera was not positioned correctly and thus its position changed from the front of the robot to the gripper, allowing its position and viewing angle to be adjusted.

4.2 Choice of Testing Environment

For testing, an open, medium-sized room was chosen. The room needed to have sufficiently large open spaces for the robot to be able to correctly plan its trajectories for the object detection phase. Due to the use of mecanum wheels and camera based object detection, the room also needed to have a mostly-smooth, flat floor with a uniform colour.

4.3 Choice of Testing Objects

Due to low video output quality of the included USB camera module, including significant motion blur and difficulties with exposure under different lighting conditions, the success rate of recognition and classification of the electronics was too low. Additionally, most electronics waste had too low of a profile to be recognised by the depth camera. As a result, it was chosen to 3D-print coloured shapes with a textured outer wall for grip. A custom dataset was created for the object classification model to improve object detection performance.

5 Results

5.1 Coverage Planners

Three path planners have been compared:

- Boustrophedon Coverage
- Morphology-based Skeleton Coverage
- Spanning Tree coverage

The Boustrophedon coverage planner package had compatibility issues with other packages used for navigation, this resulted in Boustrophedon not working on the MIRTE Master.

The morphology-based skeleton coverage planner performed well in laboratory-like environments but worse in larger open spaces. This planner allowed the robot to maneuver efficiently between objects such as tables and chairs while still being able to get into tight places. The morphology based skeleton coverage planner also resulted in the least amount of sharp turns which, due to the simple nature of the MIRTE Master odometry (wheel encoders only), helped the robot not to lose track of where it was and thus keeping the map clean. However, the skeleton planner performed slightly worse in terms of coverage performance. This resulted from the lower density of the planned path, which resulted in the robot sometimes missing the edges of larger free spaces that needs to be scanned for objects.

Figure 16: Picture of skeleton path in laboratory environment.

The spanning tree coverage planner performed well in large open spaces, but unlike the morphology based skeleton coverage planner, it struggled more with planning paths in tighter spaces such as laboratory environments, resulting in separate path loops being stitched together, and possibly missing tighter spots between the loops. Also, due to the nature of the coverage pattern, a lot of smaller radius turns are generated which resulted in the robot sometimes slowly accumulating drift in the odometry, resulting in less accurate localisation and a less accurate map.

Figure 17: Picture of spanning tree path in laboratory environment.

5.2 Object detection

5.3 Manipulation

5.3.1 Movement accuracy

5.3.2 gripping performance

6 Conclusion

This project realized an autonomously path-planning, navigating, waste-detecting and waste-processing robot based on the MIRTE Master platform. This was done by using a LiDAR, RGBD camera, robotic arm with four degrees of freedom, all run using a ROS2 software stack.

Firstly, the experiments demonstrated that the MIRTE Master platform is a decent foundation for building a custom robotic prototype such as the one made in this project.

7 Discussion

7.1 Delays

We were caught in the middle of a platform migration to a new design. This migration happened at the time that a MSc course started which used up all of the MIRTE Master robots. This meant that, for us, no robot was available from the start of our project. We also had no clarity about when we would get one and if we needed to order components ourselves and we could not order all components ourselves due to budget restrictions. In the end, we got our components in week 13 of the 16-week project period. While our software team was able to frontload the majority of software development using Gazebo simulations, this still meant that the team only had three weeks to find out all the quirks of the MIRTE Master, troubleshoot them and simultaneously integrate the code base which severely limited our ability to perform real-world quantitative and qualitative experiments.

7.2 Gazebo

Gazebo was a great tool to test our software stack, or parts of it, quickly and in an isolated manner. However, learning to work with Gazebo not only had a steep learning curve, it also required a lot of additional effort to set up a realistic environment: making custom worlds using Blender with objects that could be interacted with.

When transfering the code base from the simulation environment to the real world, many problems occurred. Firstly, the code had to be adjusted: the 'use_sim_time' had to be left unused and secondly: we found out that the sensors of the robot gave way less accurate and reliable feedback.

7.3 The MIRTE Master Platform

The MIRTE Master proved to be a viable starting platform for a cleaning robot. A lot of things were (almost) pre-configured: MoveIt was largely done, Nav2 for MIRTE was there, Gazebo already existed, configuration files for the robot were by and large integrated and operational and all the calibration and control functioned.

However, it seemed to us that some parts of the design were not tested as thoroughly as expected before integration within the MIRTE Master platform. We mainly encountered problems with the perception components. Firstly, the RGBD camera was mounted in the front, near the bottom of the MIRTE Master which resulted in the depth camera not being able to create a usable depth image for the pointcloud. We found out that this was happening by using the OrbbecViewer tool and we tested for different mounting positions. From our testing, we concluded that the depth camera should never have been mounted low to the ground. Secondly, the included USB camera module was excluded from our final design because of poor performance: the camera had too much blur for use while moving and the FPS dropped severely in different lighting and background conditions.

The final major issue that we encountered was the time-synchronization between the MIRTE Master and other laptops. We quickly discovered that the MIRTE Master's OrangePi 3B was not up to the task of running the software stack on its own, so we outsourced the processing power to the laptop of one of the team members. For this to work correctly, time needed to be synchronised between the two machines. The MIRTE Master does not possess an accurate clock and when a laptop is directly connected to the MIRTE Master, no internet connection is available. This resulted in us trying to manually sync the clocks between devices, which did not have a high success-rate. Later, a custom implementation was made with Chrony which automatically synced the time between devices within tolerances.

8 Appendix

8.1 MIRTE Modifications and Setup Guide

Clock Synchronization To syncronise the two machines' clocks, use the following command:

```
sudo date -s "date_and_time"
```

example:

```
sudo date -s "2026 -01 -01 12:01:01"
```

Install image pipeline:

```
sudo apt install ros -humble -image -pipeline
```

---

mirte-gazebo/launch/gazebo_mirte_world_generated.launch.xml

Add verbosity and add to gazebo model path

```
<launch>
<set_env name="GAZEBO_MODEL_PATH" value="$(env GAZEBO_MODEL_PATH ''):$(find -pkg -share mirte_gazebo)
<set_env name="GAZEBO_MEDIA_PATH" value="$(env GAZEBO_MEDIA_PATH ''):$(find -pkg -share mirte_gazebo)

<! - - ////////////////////////////////////////////////////////////////////////////////////////////
<set_env name="GAZEBO_MODEL_PATH" value="$(env GAZEBO_MODEL_PATH ''):$(find -pkg -share mirte_lc_gaze
<! - - ////////////////////////////////////////////////////////////////////////////////////////////

<arg name="gui" default="true"/>
<arg name="generated_world" default="$(find -pkg -share mirte_gazebo)/worlds/robocupjunior_simple.wo
<include file="$(find -pkg -share gazebo_ros)/launch/gazebo.launch.py">
    <arg name="world" value="$(var generated_world)"/>
    <arg name="gui" value="$(var gui)"/>

<! - - ////////////////////////////// - ->
    <arg name="verbose" value="true"/>
<! - - ////////////////////////////// - ->

</include>
</launch>
```

---

mirte-ros-packages/mirte_description/urdf/ultrasonic.urdf

Gazebo publishes invalid ranges if the range is larger than max

```
<gazebo reference="${name}_sonar">
  <sensor type="ray" name="range_sensor_${name}">
    <visualize>${visualize_sensors}</visualize>
      <ray>
        <scan>
          <horizontal>
            <! - - for some reason 1 does not work, we neex 2x2 - ->
            <samples>2</samples>
            <resolution>1</resolution>
            <min_angle> -0.26</min_angle>
```

```
            <max_angle>0.26</max_angle>
          </horizontal>
        <vertical>
          <samples>2</samples>
          <resolution>1</resolution>
          <min_angle> -0.014835</min_angle>
          <max_angle>0.014835</max_angle>
        </vertical>
      </scan>
      <range>
        <min>0.03</min>

        <! - -/////////////// - ->
        <max>100</max>
        <! - -/////////////// - ->

        <resolution>0.01</resolution>
      </range>
```

---

mirte-ros-packages/mirte_description/urdf/arm.urdf.xacro

Rename mimic joints with `_mimic`

```
_Gripper_joint_r_mimic
gripper_link_joint_l_mimic
_gripper_link_joint_r_mimic
```

---

mirte-ros-packages/mirte_description/mirte_master_description/urdf/arm.xacro

Replace the joint limit of the `shoulder_lift_joint` (line 55) to:

```
<limit
    lower="${ -pi + 0.2}"
    upper="${pi - 0.2}"
    effort="2.0"
    velocity="2.0"
/>
```

---

mirte-ros-packages/mirte_moveit_config/config/mirte_master.srdf

Add group states for the `mirte_arm` move group Copy the following lines exactly and place them under the "home" move group (line 29):

```
<group_state name="place_right" group="mirte_arm">
<joint name="shoulder_pan_joint" value=" -2.88"/>
<joint name="shoulder_lift_joint" value="0.262"/>
<joint name="elbow_joint" value=" -1.5"/>
<joint name="wrist_joint" value=" -1.5"/>
</group_state>
```

```
<group_state name="place_left" group="mirte_arm">
<joint name="shoulder_pan_joint" value="2.88"/>
<joint name="shoulder_lift_joint" value="0.262"/>
<joint name="elbow_joint" value=" -1.5"/>
<joint name="wrist_joint" value=" -1.5"/>
</group_state>
<group_state name="standby" group="mirte_arm">
<joint name="shoulder_pan_joint" value="0.0"/>
<joint name="shoulder_lift_joint" value=" -0.5"/>
<joint name="elbow_joint" value=" -1.3"/>
<joint name="wrist_joint" value=" -1.3"/>
</group_state>
<group_state name="vigilant" group="mirte_arm">
<joint name="shoulder_pan_joint" value="0.0"/>
<joint name="shoulder_lift_joint" value=" -0.5"/>
<joint name="elbow_joint" value=" -1.3"/>
<joint name="wrist_joint" value=" -0.4"/>
</group_state>
```

Add Gazebo Camera

```
<joint name="gripper_camera_rgb_optical_joint" type="fixed">
    <origin xyz="0 0 0" rpy="1.5707963267949 -1.5707963267949 0" />
    <parent link="gripper_camera_rgb_frame" />
    <child link="gripper_camera_rgb_optical_frame" />
</joint>

<link name="gripper_camera_rgb_optical_frame" />

<gazebo reference="gripper_camera_rgb_optical_frame">
    <sensor name="gripper_camera_sensor" type="camera">
        <always_on>true</always_on>
        <visualize>false</visualize>
        <update_rate>30</update_rate>
        <camera name="gripper_camera">
            <horizontal_fov>1.047</horizontal_fov>
            <image>
                <width>640</width>
                <height>480</height>
                <format>R8G8B8</format>
            </image>
            <clip>
                <near>0.02</near>
                <far>6</far>
            </clip>
            <distortion>
                <k1>0.0</k1>
                <k2>0.0</k2>
                <k3>0.0</k3>
                <p1>0.0</p1>
                <p2>0.0</p2>
            </distortion>
        </camera>
        <ros>
            <namespace>gripper_camera</namespace>
            <imageTopicName>image_raw</imageTopicName>
```

```
            <cameraInfoTopicName>camera_info</cameraInfoTopicName>
        </ros>
        <plugin name="gripper_camera_controller" filename="libgazebo_ros_camera.so">
            <camera_name>gripper_camera</camera_name>
        </plugin>
    </sensor>
</gazebo>

<gazebo>
    <plugin filename="libgazebo_grasp_fix.so" name="gazebo_grasp_fix">
    <arm>
        <arm_name>finger_gripper</arm_name>
        <palm_link>wrist</palm_link>
        <gripper_link>gripper</gripper_link>
        <gripper_link>_Gripper_r</gripper_link>
        <gripper_link>_gripper_link_l</gripper_link>
        <gripper_link>gripper_finger_l</gripper_link>
        <gripper_link>_gripper_link_r</gripper_link>
        <gripper_link>gripper_finger_r</gripper_link>
    </arm>
    <forces_angle_tolerance>100</forces_angle_tolerance>
    <update_rate>4</update_rate>
    <grip_count_threshold>4</grip_count_threshold>
    <max_grip_count>8</max_grip_count>
    <release_tolerance>0.005</release_tolerance>
    <disable_collisions_on_attach>false</disable_collisions_on_attach>
    </plugin>
</gazebo>
```

8.1.1 Setting up the MIRTE Master

Insert an SD card with an image into the MIRTE. Wait for flashing to conclude.

Turn on the MIRTE Master and wait for the hotspot to turn on. Connect to the MIRTE's wifi.

On your pc, insert the command:

```
ssh mirte@mirte_local
```

and insert the password:

```
mirte_mirte
```

If necessary, switch motor pins in:

```
home/mirte/mirte_ws/src/mirte -ros -packages/mirte_bringup/telemetrix_config/mirte_master_config.yaml
```

On the MIRTE, change the PID values in:

```
opt/ros/humble/share/mirte_base_control/config/mirte_base_control.yaml
```

using:

```
sudo nano
```

Set:

```
P = 0.5, I = 2.0, D = 0.0
```

for all motors.

On the MIRTE, execute the following commands:

```
ros2 run mirte_test mirte_master_set_voltage_ranges
ros2 run mirte_test mirte_master_calibrate
```

Turn off ROS2 on the MIRTE and reassemble the arm components so they match the "home" pose.

8.2 Cheatsheet

8.3 README

This is the TU Delft starterkit for open publishing with Jupyter Book. It provides a skeleton with basic settings to have a head start with your thesis / project.

It includes:

- a github deploy file taking care of deploying the website and building a pdf
- basic content files that can be altered to fit your project
- yml files in the root folder to configure the book and the pdf build

8.3.1 Where to start

Already know about Jupyter Book, familiar with GitHub or GitLab, Markdown... start with cloning the starterkit and start writing working on your project. New to the ecosystem? Start with the manual, and then move on to the starterkit.

8.3.2 Purpose

The purpose of this project and this manual is to enable others to use Jupyter Book for writing and publishing their own scientific and educational content. We hope to lower the technical barrier for researchers and educators to create and share their own resources, and to promote open science and open education practices.

8.3.3 License

8.3.4 Errors

If you find any errors in the content, please report them by opening an issue on the GitHub repository.

8.3.5 Funding

This project has received funding from the Delft University of Technology Open Science Fund (2025-2026).

8.3.6 Identifier

TUDJBOSSTARTERKIT

References

A. J. Becoy, K. Khomenko, L. Peternel, and R. T. Rajan. Autonomous navigation of quadrupeds using coverage path planning with morphological skeleton maps. *Frontiers in Robotics and AI*, Volume 12 - 2025, 2025. ISSN 2296-9144. doi:10.3389/frobt.2025.1601862. URL https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1601862.

D. Coleman, I. A. Sucan, S. Chitta, and N. Correll. Reducing the Barrier to Entry of Complex Robotic Software: a MoveIt! Case Study. *Journal of Software Engineering for Robotics*, 5(1):3–16, 2014. doi:10.6092/JOSER_2014_05_01_p3.

M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, KDD'96, pages 226–231, Portland, Oregon, 1996. AAAI Press.

M. A. Fischler and R. C. Bolles. Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography. *Commun. ACM*, 24(6):381–395, 6 1981. ISSN 0001-0782. doi:10.1145/358669.358692. URL https://doi.org/10.1145/358669.358692.

Y. Gabriely and E. Rimon. Spanning-tree based coverage of continuous areas by a mobile robot. In *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*, volume 2, pages 1927–1933 vol.2, 2001. doi:10.1109/ROBOT.2001.932890.

A. Limaran, A. Wicaksono, and P. Herwanto. Performance enhancement of embedded object detection via neural hardware acceleration. *TELKOMNIKA (Telecommunication Computing Electronics and Control)*, 24: 126, 2 2026. doi:10.12928/telkomnika.v24i1.27448.

S. Macenski and I. Jambrecic. Slam Toolbox: Slam for the dynamic world. *Journal of Open Source Software*, 6(61):2783, 2021. doi:10.21105/joss.02783. URL https://doi.org/10.21105/joss.02783.

S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The Marathon 2: A Navigation System. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. URL https://github.com/ros-planning/navigation2.

S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. W. . Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66):eabm6074, 2022. doi:10.1126/scirobotics.abm6074. URL https://www.science.org/doi/abs/10.1126/scirobotics.abm6074.

H. Niu, X. Ji, L. Zhang, F. Wen, R. Ying, and P. Liu. A Skeleton-Based Topological Planner for Exploration in Complex Unknown Environments. https://arxiv.org/abs/2412.13664, 2025. URL https://arxiv.org/abs/2412.13664.

R. Sapkota and M. Karkee. Ultralytics YOLO Evolution: An Overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 Object Detectors for Computer Vision and Pattern Recognition. 2026. URL https://arxiv.org/abs/2510.09653.

R. Sapkota, R. H. Cheppally, A. Sharda, and M. Karkee. Yolo26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection. 2026. URL https://arxiv.org/abs/2509.25164.